PROJET TRACKIT

16 — Stratégie de sécurité

Réflexion critique et documentée des besoins de sécurisation de l'application web de gestion des anomalies informatiques

Sommaire

1. Objet, périmètre et méthode
2. Biens à protéger et analyse des risques
3. Principes de Security by Design appliqués
4. Authentification et récupération des identifiants
5. Autorisation et accès aux ressources
6. Protection des données personnelles
7. Sécurité des échanges et de la configuration
8. Sécurité des pièces jointes
9. Journalisation et détection d'intrusion
10. Plan de reprise d'activité
11. Revue des dix principaux risques OWASP
12. Synthèse des écarts et plan d'action
13. Conclusion

1. Objet, périmètre et méthode

1.1 Objet du document

Ce document formalise la stratégie de sécurité de TrackIT, application web de déclaration, de suivi et de traitement des anomalies informatiques. Il ne se limite pas à énumérer de bonnes pratiques : il confronte les besoins de sécurisation aux mesures réellement implémentées dans l'application, distingue ce qui est en place de ce qui ne l'est pas, et hiérarchise les actions restantes.

Chaque affirmation portant sur une mesure existante est vérifiable dans le code source. Les écarts sont signalés explicitement par un encadré, afin qu'ils ne se confondent pas avec les dispositifs opérationnels.

1.2 Périmètre

Le périmètre couvre les trois composants de la solution :

- l'**API REST** (Spring Boot 3.3.5, Java 17), qui porte l'ensemble des règles d'authentification, d'autorisation et de validation — 61 opérations exposées ;
- le **client web** (JavaScript natif, sans framework), qui consomme l'API et stocke la session de l'utilisateur ;
- la **base de données MySQL** et le **stockage des pièces jointes** sur le système de fichiers du serveur.

Sont hors périmètre la sécurité physique de l'hébergement, la sécurité du poste de travail des utilisateurs, et le traitement des données bancaires, délégué à Stripe : TrackIT ne stocke ni numéro de carte ni donnée de paiement.

1.3 Méthode

L'analyse s'appuie sur trois sources complémentaires :

- une **revue du code source** portant sur la configuration de sécurité, les filtres d'authentification, les annotations d'autorisation, la validation des entrées, le traitement des fichiers et la gestion des erreurs ;
- le référentiel **OWASP Top 10 (édition 2021)**, utilisé comme grille de couverture au chapitre 11 ;
- les principes de **Security by Design**, utilisés comme grille de conception au chapitre 3.

Les niveaux de risque sont exprimés sur une échelle à quatre degrés — faible, moyen, élevé, critique — obtenue en croisant l'impact d'un incident avec sa vraisemblance compte tenu des mesures déjà en place.

2. Biens à protéger et analyse des risques

2.1 Biens sensibles

Avant de choisir des mesures, il faut savoir ce que l'on protège. TrackIT manipule des informations qui décrivent les pannes et les faiblesses du système d'information de ses clients : leur divulgation est plus grave que celle d'un simple catalogue de données.

Bien	Nature	Sensibilité	Conséquence d'une compromission
Mots de passe	Secret d'authentification	**Critique**	Usurpation d'identité et accès à toutes les données de l'organisation
Jetons JWT	Secret de session	**Critique**	Accès complet au compte usurpé pendant la durée de validité du jeton

Bien	Nature	Sensibilité	Conséquence d'une compromission
Anomalies et commentaires	Donnée métier interne	**Élevée**	Divulgation des pannes et des vulnérabilités du SI du client
Systèmes IT déclarés	Cartographie du SI	**Élevée**	Facilite la préparation d'une attaque ciblée contre le client
Pièces jointes	Journaux, captures, traces	**Élevée**	Fuite d'adresses IP, de noms de serveurs, de messages d'erreur internes
Données personnelles	Nom, prénom, e-mail, rôle	**Élevée**	Non-conformité au RGPD et atteinte à la vie privée des salariés
Référentiels et comptes	Donnée de configuration	**Moyenne**	Désorganisation du service, attribution de droits indus
Données de paiement	Donnée financière	Sans objet	Non stockées par TrackIT : entièrement déléguées à Stripe

2.2 Sources de menace

Le modèle de menace retenu distingue cinq profils, du plus extérieur au plus interne. Le profil le plus dangereux n'est pas l'attaquant anonyme, mais l'utilisateur déjà authentifié : il dispose d'un accès légitime et cherche à en sortir.

Source	Motivation	Capacité	Scénario principal
Attaquant externe non authentifié	Revente de données, nuisance	Moyenne	Attaque en force brute sur la connexion ; recherche d'une faille d'injection
Utilisateur authentifié malveillant	Curiosité, nuisance interne	Élevée	Tentative d'élévation de privilège ou d'accès à des anomalies hors de son périmètre
Organisation cliente concurrente	Espionnage économique	Moyenne	Tentative d'accès transversal aux données d'une autre organisation
Administrateur imprudent	Erreur involontaire	Élevée	Suppression de données, attribution d'un rôle excessif, fuite d'un secret
Poste client compromis	Vol de session	Moyenne	Récupération du jeton stocké dans le navigateur de la victime

2.3 Matrice des risques

La vraisemblance est évaluée **en tenant compte des mesures déjà implémentées**. C'est pourquoi des risques intrinsèquement graves, comme l'injection SQL, ressortent en niveau faible : le mécanisme d'accès aux données les neutralise par construction.

Réf.	Risque	Impact	Vraisemblance	Niveau	Traitement
R1	Vol d'identifiants par force brute	**Critique**	Élevée	**Élevé**	**À traiter**
R2	Interception du jeton en transit	**Critique**	Élevée hors TLS	**Élevé**	**À traiter**
R3	Perte de données sans possibilité de reprise	**Critique**	Moyenne	**Élevé**	**À traiter**
R4	Intrusion non détectée	**Élevé**	Élevée	**Élevé**	**À traiter**
R5	Indisponibilité du compte faute de réinitialisation	**Moyen**	Certaine	**Moyen**	**À traiter**
R6	Dépôt d'un fichier malveillant	**Moyen**	Moyenne	**Moyen**	**Partiel**
R7	Fuite transversale entre organisations	**Critique**	Faible	**Moyen**	**Maîtrisé**
R8	Élévation de privilège	**Élevé**	Faible	**Faible**	**Maîtrisé**
R9	Injection SQL	**Critique**	Très faible	**Faible**	**Maîtrisé**

Réf.	Risque	Impact	Vraisemblance	Niveau	Traitement
R10	Vol de jeton par injection de script (XSS)	**Élevé**	Faible	**Faible**	**Maîtrisé**

3. Principes de Security by Design appliqués

Le Security by Design consiste à intégrer la sécurité dès la conception plutôt qu'à l'ajouter après coup. Le tableau ci-dessous confronte chaque principe à sa traduction concrète dans TrackIT — ou à son absence.

Principe	Traduction dans TrackIT	État
Défense en profondeur	Trois barrières indépendantes : contrôle du rôle par annotation, règles métier dans la couche service, filtrage par organisation dans la requête SQL	**En place**
Moindre privilège	Cinq rôles distincts ; sur 61 opérations, un Utilisateur n'en atteint que 26 et un Opérateur 34	**En place**
Sécurité par défaut	`anyRequest().authenticated()` : toute route non déclarée publique est protégée. Huit routes publiques seulement, listées explicitement	**En place**
Échec sécurisé	Un jeton illisible n'authentifie pas : la requête poursuit en anonyme et se solde par un 401. Le secret JWT est validé au démarrage : sans lui, l'application refuse de démarrer	**En place**
Surface d'attaque minimale	API sans état : ni session serveur, ni cookie d'authentification. Origines autorisées déclarées une à une	**En place**
Non-divulgation d'information	Une ressource appartenant à une autre organisation renvoie 404 et non 403, pour ne pas révéler son existence. Le message d'échec de connexion est identique que l'adresse existe ou non	**En place**
Médiation complète	Chaque requête revalide la signature du jeton **et** recharge l'utilisateur en base : un compte désactivé perd l'accès immédiatement, sans attendre l'expiration du jeton	**En place**
Séparation des secrets	Aucun secret dans le code : variables d'environnement, fichier `.env` exclu du dépôt, `.env.example` ne contenant que des valeurs factices	**En place**
Traçabilité	Historique fonctionnel horodaté et nominatif sur les anomalies ; en revanche aucun journal dédié aux événements de sécurité	**Partiel**
Limitation du débit	Aucun mécanisme de limitation du nombre de tentatives ou d'appels	**Absent**

4. Authentification et récupération des identifiants

4.1 Mécanisme retenu et justification

TrackIT authentifie ses utilisateurs par **jeton JWT signé**, et non par session serveur. Ce choix n'est pas gratuit : l'API est consommée par un client web séparé, potentiellement hébergé sur un autre domaine. Une session fondée sur un cookie aurait imposé une gestion du partage de cookies entre domaines et aurait exposé l'application aux attaques CSRF. Un jeton transmis dans l'en-tête `Authorization` n'est pas envoyé automatiquement par le navigateur : la falsification de requête inter-site devient sans objet.

La contrepartie est connue et doit être assumée : un jeton signé **ne peut pas être révoqué** avant son expiration. TrackIT atténue cette faiblesse par une mesure spécifique décrite au point 4.2.

Élément	Valeur implémentée	Appréciation
Algorithme de signature	HMAC-SHA, clé de 512 bits	Conforme à l'état de l'art
Longueur minimale de la clé	32 caractères, contrôlée au démarrage	Satisfaisant

Élément	Valeur implémentée	Appréciation
Durée de validité	480 minutes (8 heures), paramétrable	Longue ; voir 4.2
Contenu du jeton	Identifiant, e-mail, rôle, organisation	Aucune donnée secrète
Empreinte des mots de passe	BCrypt, facteur de coût 10	Conforme ; sel automatique
Transport du jeton	En-tête `Authorization: Bearer`	Conforme
Stockage côté client	`localStorage` ou `sessionStorage` selon « Se souvenir de moi »	Acceptable, mais voir 7.6

Le choix de BCrypt mérite d'être souligné. Contrairement à une empreinte SHA-256, BCrypt est **volontairement lent** et intègre un sel unique par mot de passe. Deux utilisateurs ayant choisi le même mot de passe obtiennent deux empreintes différentes, et une attaque par dictionnaire sur une base dérobée devient coûteuse.

4.2 Cycle de vie du jeton et révocation

Le filtre d'authentification ne se contente pas de vérifier la signature du jeton. À chaque requête, il **recharge l'utilisateur depuis la base** et vérifie que son compte est toujours actif. Cette décision a un coût — une lecture supplémentaire par appel — mais elle apporte deux garanties déterminantes :

- la désactivation d'un compte par un administrateur prend effet **immédiatement**, sans attendre l'expiration du jeton ;
- un changement de rôle est pris en compte à la requête suivante, car les habilitations sont relues en base et non extraites du jeton.

C'est une application directe du principe de médiation complète : l'autorisation est réévaluée à chaque accès plutôt que figée au moment de la connexion.

Limite résiduelle

Un jeton dérobé reste valable jusqu'à huit heures tant que le compte n'est pas désactivé. Il n'existe ni déconnexion côté serveur, ni liste de révocation, ni jeton de rafraîchissement de courte durée. Pour une mise en production, le schéma recommandé est un jeton d'accès de 15 minutes associé à un jeton de rafraîchissement révocable conservé en base.

4.3 Politique de mot de passe

La politique actuelle impose une longueur comprise entre 8 et 72 caractères. La borne haute n'est pas arbitraire : au-delà de 72 octets, BCrypt tronque silencieusement l'entrée, et refuser explicitement ces mots de passe évite une fausse impression de robustesse.

Le changement de mot de passe par l'utilisateur est correctement protégé : l'ancien mot de passe est exigé et vérifié, et le nouveau doit être différent de l'actuel. Un attaquant qui disposerait momentanément d'une session ouverte ne peut donc pas verrouiller le compte sans connaître le mot de passe courant.

Écart — politique trop permissive

Aucune exigence de composition, aucune vérification contre les mots de passe les plus courants, aucune détection de similarité avec l'adresse e-mail. Un mot de passe tel que « 12345678 » est accepté. **Mesure proposée :** porter le minimum à 12 caractères, refuser une liste des mots de passe les plus répandus, et afficher un indicateur de robustesse côté client. L'approche par liste de refus est préférée aux règles de composition, qui produisent des mots de passe prévisibles.

4.4 Récupération des identifiants

Le sujet est explicitement demandé par le cahier des charges, et c'est l'écart le plus visible de l'application.

Écart — fonctionnalité absente

L'API n'expose aucune opération de réinitialisation de mot de passe : le groupe « Authentification » ne contient que la connexion et l'inscription. L'interface affiche pourtant un lien « Mot de passe oublié ? » sur l'écran de connexion. Un utilisateur ayant perdu son mot de passe dépend donc entièrement d'un administrateur de son organisation, qui peut le réinitialiser via l'API d'administration. Pour un administrateur unique ayant perdu son accès, il n'existe aucune issue applicative.

La conception proposée tient en deux opérations publiques et respecte les précautions d'usage :

Étape	Opération	Règles de sécurité
1. Demande	`POST /api/v1/auth/mot-de-passe-oublie`	Réponse **toujours identique** (202 Accepted), que l'adresse existe ou non, afin de ne pas révéler les comptes enregistrés. Limitation à trois demandes par heure et par adresse
2. Envoi	Courriel contenant un lien de réinitialisation	Jeton aléatoire de 256 bits, stocké **sous forme d'empreinte** en base, valable 30 minutes, utilisable une seule fois
3. Réinitialisation	`POST /api/v1/auth/reinitialisation`	Vérification du jeton et de sa date, application de la politique de mot de passe, invalidation du jeton, journalisation de l'événement, notification à l'utilisateur

Trois précautions méritent d'être explicitées, car elles sont régulièrement omises : la réponse uniforme à l'étape 1 évite de transformer la fonction en outil d'énumération des comptes ; le stockage de l'**empreinte** du jeton empêche qu'une fuite de la base permette de réinitialiser les mots de passe ; l'usage unique bloque le rejeu d'un lien retrouvé dans l'historique d'une boîte de réception.

4.5 Protection contre les attaques en force brute

Écart — aucune limitation de débit

L'opération de connexion peut être appelée sans aucune restriction de fréquence. Le coût de calcul de BCrypt ralentit l'attaquant mais ne le bloque pas, et il protège surtout contre une attaque hors ligne après vol de la base — pas contre une attaque en ligne. Rien n'empêche aujourd'hui des milliers de tentatives sur un même compte, ni la projection d'un mot de passe courant sur tous les comptes connus. Ce point constitue, avec l'absence de chiffrement du transport, le risque le plus élevé de l'application.

La réponse proposée combine trois mécanismes complémentaires :

- **verrouillage temporaire par compte** : après cinq échecs consécutifs, le compte est bloqué quinze minutes, le compteur étant remis à zéro par une connexion réussie ;
- **limitation par adresse IP** : au plus vingt tentatives par minute, afin de contenir la projection d'un même mot de passe sur de nombreux comptes ;
- **journalisation et alerte** : chaque échec alimente le journal de sécurité décrit au chapitre 9, ce qui permet de détecter une campagne en cours.

Le verrouillage par compte seul serait insuffisant : il ouvrirait une possibilité de déni de service en bloquant volontairement le compte d'autrui. C'est la combinaison des deux premiers mécanismes, et non l'un d'eux isolément, qui constitue la mesure.

4.6 Authentification multifacteur

L'authentification à deux facteurs n'est pas implémentée et n'est pas proposée pour cette version. Ce choix doit être assumé plutôt que passé sous silence : TrackIT manipule des informations techniques sensibles, mais ni données bancaires ni données de santé. Le rapport entre le gain de sécurité et la

complexité ajoutée — gestion des secrets TOTP, codes de secours, procédure de perte du second facteur — ne le justifie pas à ce stade. Il en irait autrement pour les comptes **Administrateur**, qui disposent des droits les plus étendus ; une seconde version pourrait réserver la double authentification à ce seul rôle, ce qui limiterait la charge tout en couvrant les comptes les plus exposés.

5. Autorisation et accès aux ressources

5.1 Modèle de rôles

Cinq rôles structurent le contrôle d'accès. Le nombre d'opérations atteignables par chacun, sur les 61 exposées, traduit directement le principe du moindre privilège.

Rôle	Vocation	Opérations accessibles
Utilisateur	Déclare et suit ses propres anomalies	26 / 61
Opérateur	Prend en charge et traite les anomalies affectées	34 / 61
Responsable technique	Supervise la charge, affecte, arbitre les priorités	35 / 61
Analyste	Produit les statistiques et les rapports d'activité	30 / 61
Administrateur	Gère les comptes, les référentiels et l'abonnement	49 / 61

Aucun rôle n'accède à tout, pas même l'Administrateur : il gère les comptes et la configuration, mais ni les rapports ni la supervision — ce qui limite la portée d'une compromission du compte le plus privilégié.

Point fort

Seize tests d'autorisation automatisés vérifient ce cloisonnement et le respect des rôles à chaque exécution de la suite de tests. La sécurité n'est donc pas seulement documentée : elle est vérifiée de façon reproductible à chaque modification du code.

5.2 Une double barrière de contrôle

Deux contrôles indépendants s'appliquent successivement, et c'est leur indépendance qui fait la solidité de l'ensemble.

Première barrière : le rôle

Une annotation déclarative exige un rôle minimal, et s'exécute avant l'entrée dans la méthode : une requête mal habilitée n'atteint jamais la logique métier.

Le changement de statut, par exemple, porte l'annotation `@PreAuthorize("hasAnyRole('OPERATEUR','RESPONSABLE_TECHNIQUE','ADMINISTRATEUR')")` : les deux autres rôles sont écartés avant toute exécution.

Seconde barrière : la règle métier

Le rôle ne suffit pas : un Opérateur peut changer un statut **en général**, mais pas celui de n'importe quelle anomalie — encore faut-il qu'elle lui soit affectée et que la transition soit autorisée. Ces règles sont portées par un composant dédié de la couche service, distinct du contrôle de rôle.

Une erreur dans une annotation ne suffit donc pas à ouvrir un accès indu, et réciproquement.

5.3 Cloisonnement entre organisations

Plusieurs organisations clientes partagent la même base. Le cloisonnement repose sur une règle appliquée sans exception : **toute requête de lecture est filtrée par l'identifiant d'organisation issu du jeton**, jamais par un paramètre fourni par le client.

Un détail mérite d'être relevé : une anomalie d'une autre organisation renvoie **404 Introuvable** et non **403 Interdit**. Un 403 confirmerait son existence et permettrait de cartographier les données d'autrui par énumération ; le code le documente.

6. Protection des données personnelles

6.1 Données collectées et finalités

TrackIT traite des données à caractère personnel concernant les salariés de ses organisations clientes. Le registre ci-dessous recense ces données, leur finalité et la durée de conservation proposée.

Donnée	Finalité	Base légale	Conservation proposée
Nom, prénom	Identifier l'auteur d'une déclaration ou d'une action	Exécution du contrat	Durée du compte + 1 an
Adresse e-mail	Authentifier et notifier l'utilisateur	Exécution du contrat	Durée du compte + 1 an
Empreinte du mot de passe	Authentifier l'utilisateur	Exécution du contrat	Supprimée avec le compte
Rôle et organisation	Déterminer les droits d'accès	Intérêt légitime	Durée du compte
Historique des actions	Tracer les modifications, servir de preuve	Intérêt légitime	3 ans
Contenu des anomalies	Assurer le traitement de l'incident	Exécution du contrat	Clôture + 2 ans
Pièces jointes	Diagnostiquer l'incident	Exécution du contrat	Clôture + 2 ans

6.2 Mesures de protection en place

- **Minimisation** : aucune donnée non nécessaire n'est demandée — ni date de naissance, ni adresse postale, ni numéro de téléphone.
- **Confidentialité des secrets** : les mots de passe ne sont jamais stockés en clair ni renvoyés par l'API ; seule leur empreinte BCrypt est conservée.
- **Cloisonnement** : les données personnelles d'une organisation sont inaccessibles aux membres d'une autre, par construction.
- **Exactitude** : chaque utilisateur peut corriger lui-même son nom, son prénom et son adresse via son profil.
- **Traçabilité** : l'historique conserve l'auteur et la date de chaque modification, ce qui permet de répondre à une demande d'explication.

6.3 Écarts de conformité

Écart — droits des personnes non outillés

L'API n'expose aucune opération permettant à un utilisateur d'exporter ses données dans un format réutilisable (droit à la portabilité) ni d'en demander l'effacement (droit à l'oubli). La suppression d'un compte par un administrateur n'est pas documentée quant à son effet sur les anomalies déclarées. **Mesure proposée :** une opération d'export au format JSON, et une procédure d'anonymisation — remplacement du nom et de l'adresse par une mention « utilisateur supprimé » — préférable à une suppression pure, qui ferait perdre la cohérence de l'historique.

Écart — durées de conservation non appliquées

Les durées figurant au tableau 6.1 sont des propositions : aucun mécanisme de purge automatique n'existe. Les données sont conservées sans limite. **Mesure proposée :** une tâche planifiée mensuelle appliquant les durées retenues, assortie d'un journal des suppressions effectuées.

Un troisième point relève de l'organisation plus que de la technique : l'information des personnes. Une politique de confidentialité accessible depuis l'interface, précisant les finalités, les durées et les modalités d'exercice des droits, reste à rédiger.

7. Sécurité des échanges et de la configuration

7.1 Chiffrement du transport

Écart — l'application n'impose pas HTTPS

La configuration du serveur n'active aucun certificat TLS : l'API est servie en HTTP. Or le jeton JWT circule dans chaque requête, et le mot de passe transite en clair lors de la connexion. Sur un réseau non maîtrisé, une écoute passive suffit à les capturer. C'est, avec l'absence de limitation du débit, le risque le plus élevé identifié.

La mesure n'est pas coûteuse et relève du déploiement plutôt que du code. Trois dispositions sont proposées :

- placer l'application derrière un **terminateur TLS** (Nginx ou Traefik) muni d'un certificat Let's Encrypt renouvelé automatiquement ;
- **rediriger systématiquement** le trafic HTTP vers HTTPS ;
- activer l'en-tête **HSTS** (`Strict-Transport-Security`), qui interdit au navigateur de retenter une connexion non chiffrée pendant la durée indiquée.

Il faut noter que HSTS n'a de sens qu'une fois HTTPS opérationnel : activé seul, il rendrait l'application inaccessible. L'ordre des opérations fait ici partie de la mesure.

7.2 Partage des ressources entre origines (CORS)

Le client web étant servi depuis une origine distincte de l'API, une politique CORS est nécessaire. Elle est volontairement restrictive : les origines autorisées sont déclarées une à une dans la configuration et lues depuis l'environnement, les méthodes HTTP sont énumérées explicitement, et seuls deux en-têtes sont acceptés en requête. Le caractère générique `*`, qui autoriserait n'importe quel site à appeler l'API avec les droits de l'utilisateur, n'est utilisé nulle part.

7.3 Falsification de requête inter-site (CSRF)

La protection CSRF de Spring Security est désactivée. Ce point attire généralement l'attention d'un auditeur, et il mérite une justification explicite plutôt qu'une mention en passant.

Une attaque CSRF repose sur le fait que le navigateur joint **automatiquement** les identifiants de session à une requête vers un domaine donné — typiquement un cookie. TrackIT n'utilise ni cookie d'authentification ni session serveur : le jeton doit être ajouté explicitement par le code JavaScript dans l'en-tête `Authorization`. Un site tiers ne peut pas provoquer cet ajout. La condition même de l'attaque est donc absente, et maintenir la protection CSRF n'apporterait aucune sécurité supplémentaire tout en compliquant l'usage de l'API.

Cette justification cesserait d'être valable si une future version stockait le jeton dans un cookie. La décision est donc **conditionnelle au modèle d'authentification** et devra être réexaminée si celui-ci évolue.

7.4 En-têtes de sécurité

En-tête	Valeur	Effet	État
`X-Frame-Options`	DENY	Interdit l'inclusion dans une iframe (clickjacking)	**En place**
`X-Content-Type-Options`	nosniff	Empêche le navigateur de deviner le type de contenu	**En place**
`Cache-Control`	no-store	Évite la mise en cache des réponses authentifiées	**En place**
`Strict-Transport-Security`	—	Impose HTTPS pour les visites suivantes	**Absent**
`Content-Security-Policy`	—	Restreint les sources de scripts exécutables	**Absent**

Les trois premiers en-têtes proviennent des réglages par défaut de Spring Security et sont effectivement présents dans les réponses. Les deux derniers supposent une configuration explicite. Une politique CSP même minimale constituerait une seconde ligne de défense utile contre l'injection de script, en interdisant l'exécution de code provenant d'une origine non déclarée.

7.5 Gestion des secrets

Aucun secret n'est inscrit dans le code source. Le mot de passe de la base, la clé de signature des jetons et les clés Stripe sont lus depuis l'environnement. Le fichier `.env` qui les porte est exclu du dépôt, et un `.env.example` documente les variables attendues avec des valeurs factices.

La configuration applique en outre une vérification au démarrage : la clé de signature est déclarée obligatoire et doit compter au moins 32 caractères. En son absence, l'application **refuse de démarrer** plutôt que de se rabattre sur une valeur par défaut — c'est une application du principe d'échec sécurisé, et cela écarte le scénario classique d'une mise en production avec une clé de développement.

Vérification effectuée

Une recherche des valeurs du fichier `.env` dans l'ensemble des fichiers versionnés n'a révélé **aucune recopie** du mot de passe de base ni de la clé de signature. Le dépôt peut être publié sans exposer de secret.

7.6 Stockage de la session côté client

Le jeton est conservé dans `localStorage` ou `sessionStorage` selon que l'utilisateur a coché « Se souvenir de moi ». Ce stockage est lisible par tout script s'exécutant dans la page : en cas d'injection de script, le jeton peut être exfiltré.

Le risque est cependant **contenu**, parce que la condition préalable — une faille XSS — n'a pas été trouvée. Le client échappe systématiquement les données provenant du serveur avant insertion dans le HTML, au moyen d'une fonction d'échappement appliquée aux caractères `&`, `<`, `>`, `"` et `'`. Les quelques interpolations qui ne l'appellent pas directement ont été examinées une à une : elles passent soit par un générateur de champ de formulaire qui échappe lui-même, soit par `textContent`, soit par le titre de la page — aucune n'écrit de donnée utilisateur non échappée dans le document.

L'alternative la plus robuste resterait un cookie `HttpOnly`, `Secure` et `SameSite`, inaccessible au JavaScript ; elle imposerait en contrepartie de rétablir une protection CSRF. Le choix actuel est défendable tant que la discipline d'échappement est maintenue : c'est un point de vigilance pour toute évolution du client.

8. Sécurité des pièces jointes

Le dépôt de fichiers est l'une des fonctions les plus exposées d'une application web : elle accepte des données arbitraires et les écrit sur le serveur. Les contrôles implémentés sont détaillés ci-dessous, avec leurs limites.

Contrôle	Mise en œuvre	Appréciation
Extensions autorisées	Liste blanche de 13 formats (pdf, png, jpg, jpeg, gif, txt, log, csv, json, xml, docx, xlsx, zip) ; tout autre format est refusé	Liste blanche plutôt que liste noire : approche correcte
Taille maximale	10 Mo par fichier, contrôlés à deux niveaux : conteneur web et couche métier	Double contrôle : une requête volumineuse est rejetée avant d'être lue entièrement
Nom de fichier	Chemin retiré, caractères spéciaux remplacés, longueur plafonnée à 150 caractères, noms commençant par un point refusés	Neutralise les noms conçus pour écraser un fichier existant
Traversée de répertoire	Le chemin final est normalisé puis comparé au répertoire racine ; toute sortie est rejetée	Protection explicite contre `../` ; mesure essentielle
Collision de noms	Chaque fichier est préfixé par un identifiant aléatoire unique	Deux fichiers homonymes ne peuvent pas s'écraser
Type MIME enregistré	Déterminé par le serveur à partir de la liste blanche, jamais repris de la requête	Un client ne peut pas imposer un type trompeur
Contrôle d'accès au téléchargement	Les droits sur l'anomalie porteuse sont revérifiés avant toute lecture du fichier	Pas d'accès direct par l'URL seule

Limites identifiées

Le type réel du fichier n'est pas vérifié : un exécutable renommé en `.png` est accepté. Aucune analyse antivirale n'est effectuée. Le format `zip` est autorisé sans inspection de son contenu, ce qui laisse place à une archive piégée. **Mesures proposées :** contrôler les premiers octets du fichier pour confirmer le format annoncé, brancher une analyse antivirale sur le flux déposé, et servir les fichiers depuis un domaine distinct afin qu'un contenu malveillant ne s'exécute pas dans le contexte de l'application.

La gravité reste mesurée : les fichiers sont systématiquement servis en pièce jointe et ne sont jamais interprétés par le serveur. Un fichier piégé devrait être téléchargé puis ouvert volontairement par un utilisateur pour produire un effet.

9. Journalisation et détection d'intrusion

9.1 Traçabilité existante

TrackIT dispose d'une traçabilité **fonctionnelle** solide. Chaque modification significative d'une anomalie — création, changement de statut, affectation, changement de priorité, ajout de pièce jointe — est enregistrée dans un historique horodaté et nominatif, consultable depuis la fiche. Les erreurs inattendues sont consignées côté serveur avec leur contexte, sans jamais être renvoyées au client.

9.2 Écart : absence de journal de sécurité

Écart — les événements de sécurité ne sont pas journalisés

L'historique trace ce que font les utilisateurs **autorisés** sur les anomalies. Il ne trace pas les événements de sécurité : échecs de connexion, refus d'autorisation, tentatives d'accès à une ressource d'une autre organisation, jetons invalides. Ces événements sont précisément ceux qui signalent une attaque en cours. En l'état, une campagne de force brute ou une tentative méthodique d'élévation de privilège ne laisserait aucune trace exploitable.

9.3 Système de détection d'intrusion proposé

La détection d'intrusion est abordée ici à deux niveaux. Le niveau réseau relève de l'hébergement : un outil tel que Fail2ban, branché sur les journaux du serveur frontal, bloque au niveau du pare-feu les

adresses à l'origine de tentatives répétées. Le niveau applicatif, qui relève de TrackIT, repose sur un journal dédié et sur des règles de détection fondées sur des seuils.

Le préalable est un **journal de sécurité distinct** du journal applicatif, où chaque entrée porte l'horodatage, le type d'événement, l'adresse IP, l'utilisateur concerné lorsqu'il est connu, la ressource visée et le résultat. Les règles suivantes sont proposées :

Événement détecté	Seuil	Interprétation	Réaction
Échecs de connexion sur un même compte	5 en 15 min	Tentative de deviner un mot de passe	Verrouillage temporaire + alerte
Échecs de connexion depuis une même IP	20 en 1 min	Projection d'un mot de passe sur plusieurs comptes	Blocage de l'adresse + alerte
Refus d'autorisation (403) par un compte	10 en 5 min	Exploration des droits, tentative d'élévation	Alerte à l'administrateur
Accès refusés à d'autres organisations	3 en 1 h	Tentative de franchir le cloisonnement	Alerte prioritaire + revue du compte
Jetons invalides ou falsifiés	5 en 10 min	Manipulation de jeton	Alerte
Dépôts de fichiers refusés	10 en 10 min	Recherche d'une faille dans le filtrage	Alerte
Connexion réussie depuis une IP inhabituelle	1	Possible usurpation	Notification à l'utilisateur concerné

La mise en œuvre peut être progressive, et c'est ce qui la rend réaliste dans le cadre d'un projet de fin d'études. Une première étape consiste à écrire le journal de sécurité et à exposer un tableau de bord réservé à l'Administrateur. Une deuxième étape ajoute les seuils et les alertes par courriel. Une troisième, si le contexte le justifie, exporte le journal vers un outil de corrélation externe.

10. Plan de reprise d'activité

10.1 Objectifs de reprise

Un plan de reprise se définit par deux objectifs chiffrés, qui conditionnent tous les choix techniques : la durée maximale d'interruption admissible et la quantité maximale de données que l'on accepte de perdre.

Objectif	Valeur retenue	Justification
RTO — durée maximale d'interruption	4 heures	TrackIT n'est pas un outil de production temps réel : une indisponibilité d'une demi-journée dégrade le suivi des incidents sans bloquer l'activité du client
RPO — perte de données maximale	24 heures	Une sauvegarde quotidienne est acceptable au regard du volume de déclarations ; une anomalie perdue peut être [illegible]
Durée de rétention des sauvegardes	30 jours glissants	Permet de revenir avant une corruption détectée tardivement

10.2 Sauvegardes

Écart — aucune sauvegarde automatisée

Le projet contient un export complet de la base, produit manuellement. Il n'existe ni planification, ni copie hors du serveur, ni test de restauration. Une défaillance du disque ou une suppression accidentelle entraînerait aujourd'hui une perte définitive.

Le dispositif proposé repose sur la règle dite **3-2-1** : trois copies des données, sur deux supports différents, dont une hors site.

Élément	Fréquence	Rétention	Emplacement
Base MySQL — sauvegarde complète	Quotidienne (2 h du matin)	30 jours	Serveur + copie chiffrée hors site
Base MySQL — journaux binaires	Continue	7 jours	Permet un retour à un instant précis
Pièces jointes et rapports	Quotidienne, incrémentale	30 jours	Copie chiffrée hors site
Configuration et fichier `.env`	À chaque modification	Permanente	Coffre à secrets, hors dépôt
Code source	À chaque livraison	Permanente	Dépôt Git distant

Deux précisions conditionnent l'efficacité réelle du dispositif. Les sauvegardes doivent être **chiffrées** avant transfert hors site, faute de quoi elles deviennent elles-mêmes un point de fuite — une sauvegarde contient l'intégralité des données de tous les clients. Et elles doivent être **testées** : une sauvegarde dont la restauration n'a jamais été vérifiée ne constitue pas une garantie. Un test trimestriel de restauration complète sur un environnement séparé est proposé, avec consignation de la durée obtenue et comparaison au RTO visé.

10.3 Procédure de restauration

Étape	Action	Durée estimée
1	Déclarer l'incident et interrompre l'accès utilisateur	15 min
2	Qualifier la panne : matériel, corruption, erreur humaine, intrusion	30 min
3	Préparer le serveur de reprise et vérifier les prérequis	45 min
4	Restaurer la dernière sauvegarde de base valide	45 min
5	Rejouer les journaux binaires jusqu'au point choisi	30 min
6	Restaurer les pièces jointes et la configuration	30 min
7	Contrôler l'intégrité : connexion, volumétrie, cohérence des historiques	30 min
8	Rouvrir le service et informer les organisations clientes	15 min

Le cumul atteint environ trois heures et demie, ce qui laisse une marge faible mais réelle au regard du RTO de quatre heures. En cas d'intrusion avérée, l'étape 2 doit s'accompagner d'une rotation de l'ensemble des secrets — clé de signature des jetons, mots de passe de base, clés Stripe — car une sauvegarde restaurée telle quelle réintroduirait des secrets potentiellement compromis.

10.4 Déploiement et retour arrière

La reprise ne concerne pas que les sinistres : une livraison défectueuse doit pouvoir être annulée. Deux éléments y contribuent déjà. Le schéma de la base est géré par **migrations versionnées**, ce qui rend son état reproductible et traçable. Et la configuration impose à l'outil de persistance de **valider** le schéma au démarrage sans jamais le modifier : une incohérence entre le code et la base est détectée immédiatement, au lieu d'entraîner une altération silencieuse des données.

Il reste à formaliser la procédure de retour arrière : conservation de la version précédente de l'application, sauvegarde systématique de la base avant toute migration, et vérification après livraison.

11. Revue des dix principaux risques OWASP

L'OWASP Top 10 recense les dix catégories de failles les plus répandues dans les applications web. Chaque catégorie est reprise ci-dessous, confrontée à l'exposition réelle de TrackIT et aux mesures en place. Le risque résiduel est ce qui subsiste **après** application de ces mesures.

A01 — Contrôle d'accès défaillant

Exposition : forte. C'est le risque principal d'une API multi-rôles et multi-organisations. Mesures en place : contrôle du rôle par annotation sur chaque opération sensible, règles métier dans la couche service, filtrage par organisation appliqué à toutes les requêtes de lecture, réponse 404 plutôt que 403 pour ne pas révéler l'existence des ressources d'autrui, et seize tests automatisés vérifiant ces règles. **Risque résiduel : faible.**

A02 — Défaillances cryptographiques

Exposition : moyenne. Mesures en place : mots de passe protégés par BCrypt avec sel automatique, jetons signés avec une clé de 512 bits, secrets externalisés et absents du dépôt, aucune donnée confidentielle placée dans le jeton. En revanche, les données transitent en clair tant que le chiffrement du transport n'est pas activé. **Risque résiduel : élevé jusqu'à l'activation de HTTPS, faible ensuite.**

A03 — Injection

Exposition : moyenne. Mesures en place : l'accès aux données passe exclusivement par une couche de persistance qui construit des requêtes paramétrées, y compris pour la recherche multicritère ; aucune concaténation de chaîne SQL n'a été trouvée. Toutes les entrées sont validées de façon déclarative, et les paramètres de pagination sont bornés. Côté client, les données du serveur sont échappées avant insertion dans la page. **Risque résiduel : faible.**

A04 — Conception non sécurisée

Exposition : moyenne. Mesures en place : modèle de rôles pensé en amont, cycle de vie des anomalies contrôlé par des transitions explicites, cloisonnement multi-organisations intégré au modèle de données. Faiblesses de conception subsistantes : absence de parcours de récupération de mot de passe et absence de limitation du débit, deux fonctions qui relèvent de la conception et non du réglage. **Risque résiduel : moyen.**

A05 — Mauvaise configuration de sécurité

Exposition : moyenne. Mesures en place : aucune trace d'exception n'est renvoyée au client, les messages d'erreur sont uniformisés, le schéma de base est validé et non modifié au démarrage, les en-têtes de protection par défaut sont actifs, et la clé de signature est obligatoire. Manquent une politique de sécurité du contenu et l'en-tête HSTS. **Risque résiduel : moyen.**

A06 — Composants vulnérables ou obsolètes

Exposition : moyenne. L'application s'appuie sur un socle récent et maintenu (Spring Boot 3.3.5, Java 17), avec des versions de bibliothèques explicitement fixées, ce qui rend les livraisons reproductibles. En revanche, aucune analyse automatisée des vulnérabilités connues n'est effectuée : une faille publiée sur une dépendance ne serait pas détectée. **Mesure proposée :** intégrer un contrôle de dépendances à chaque livraison. **Risque résiduel : moyen.**

A07 — Défaillances d'identification et d'authentification

Exposition : forte. Mesures en place : empreintes BCrypt, message d'échec identique que le compte existe ou non, prise en compte immédiate d'un compte désactivé, mot de passe actuel exigé pour tout changement. Faiblesses : aucune limitation des tentatives, politique de mot de passe permissive, absence de second facteur, absence de réinitialisation autonome. **Risque résiduel : élevé.** C'est, avec A02, la catégorie la plus exposée de l'application.

A08 — Défaut d'intégrité des données et des logiciels

Exposition : faible. Mesures en place : la signature des notifications de paiement est vérifiée avant tout traitement, ce qui empêche un tiers de simuler un paiement accepté ; les évolutions du schéma passent par des migrations versionnées et ordonnées ; les dépendances proviennent du dépôt central Maven avec des versions fixées. **Risque résiduel : faible.**

A09 — Carences de journalisation et de supervision

Exposition : forte. L'historique fonctionnel trace les actions des utilisateurs autorisés, mais aucun journal ne recense les événements de sécurité et aucune alerte n'est déclenchée. Une intrusion pourrait rester indétectée. C'est l'objet du chapitre 9. **Risque résiduel : élevé.**

A10 — Falsification de requête côté serveur

Exposition : très faible. L'application n'effectue aucun appel réseau vers une URL fournie par un utilisateur. Les seuls appels sortants visent Stripe, dont l'adresse est fixée dans la configuration. **Risque résiduel : très faible.**

Synthèse de la couverture

Catégorie OWASP	Exposition	Risque résiduel
A01 — Contrôle d'accès défaillant	Forte	**Faible**
A02 — Défaillances cryptographiques	Moyenne	**Élevé (sans HTTPS)**
A03 — Injection	Moyenne	**Faible**
A04 — Conception non sécurisée	Moyenne	**Moyen**
A05 — Mauvaise configuration	Moyenne	**Moyen**
A06 — Composants vulnérables	Moyenne	**Moyen**
A07 — Défaillances d'authentification	Forte	**Élevé**
A08 — Défaut d'intégrité	Faible	**Faible**
A09 — Carences de journalisation	Forte	**Élevé**
A10 — Falsification côté serveur	Très faible	**Très faible**

Trois catégories sur dix présentent un risque résiduel élevé. Elles ont un point commun qui mérite d'être relevé : aucune ne provient d'une erreur dans le code écrit, mais toutes trois d'une **fonction non encore développée** ou d'un **réglage de déploiement non effectué**. Les mécanismes implémentés sont corrects ; ce sont les dispositifs périphériques — chiffrement du transport, limitation du débit, journal de sécurité — qui manquent.

12. Synthèse des écarts et plan d'action

12.1 Ce qui est acquis

La synthèse serait incomplète si elle ne retenait que les manques. Les acquis suivants constituent le socle du dispositif :

- un **contrôle d'accès à deux niveaux**, rôle puis règle métier, vérifié par des tests ;
- un **cloisonnement multi-organisations** sans exception, non divulgant ;
- une **absence de faille d'injection** sur le périmètre audité, SQL comme script ;
- un **traitement des fichiers déposés** couvrant les contrôles essentiels ;
- une **gestion des secrets** conforme, rien de sensible dans le code ni le dépôt ;
- une **gestion des erreurs** sans trace d'exécution ni fuite sur l'existence des comptes.

12.2 Écarts et plan d'action

Les écarts relevés tout au long du document sont rassemblés ici et ordonnés par priorité. L'effort est estimé en jours-homme pour un développeur connaissant l'application.

L'ordre suit le rapport entre le gain et l'effort. Les deux premières actions ferment les voies d'attaque les plus directes — écoute du réseau et essai massif de mots de passe — pour trois jours de travail. P3 à P5 relèvent de la continuité de service et de la détection ; les suivantes consolident un niveau déjà acceptable.

Priorité	Écart	Action corrective	Chap.	Effort
P1 — Critique	Transport non chiffré	Terminateur TLS, redirection HTTP vers HTTPS, en-tête HSTS	7.1	1 j
P2 — Critique	Aucune limitation des tentatives de connexion	Verrouillage temporaire par compte et limitation par adresse IP	4.5	2 j
P3 — Élevé	Aucune sauvegarde automatisée	Sauvegarde quotidienne chiffrée hors site et test de restauration	10.2	2 j
P4 — Élevé	Événements de sécurité non journalisés	Journal de sécurité dédié et tableau de bord pour l'Administrateur	9.2	3 j
P5 — Élevé	Pas de récupération de mot de passe	Deux opérations publiques avec jeton à usage unique de 30 minutes	4.4	3 j
P6 — Moyen	Politique de mot de passe permissive	Minimum de 12 caractères et liste de refus des mots de passe courants	4.3	1 j
P7 — Moyen	Dépendances non analysées	Contrôle automatisé des vulnérabilités à chaque livraison	11	0,5 j
P8 — Moyen	Contenu réel des fichiers non vérifié	Contrôle des premiers octets et analyse antivirale	8	2 j
P9 — Moyen	Politique de sécurité du contenu absente	En-tête CSP restreignant les sources de scripts	7.4	1 j
P10 — Faible	Droits RGPD non outillés	Opérations d'export et d'anonymisation, purge planifiée	6.3	3 j
P11 — Faible	Jeton non révocable	Jeton d'accès court et jeton de rafraîchissement révocable	4.2	3 j

13. Conclusion

La sécurité de TrackIT repose sur un socle applicatif solide. Les mécanismes qui demandent le plus de rigueur à l'écriture — contrôle d'accès, cloisonnement entre clients, validation des entrées, traitement des fichiers déposés, gestion des secrets — sont en place et, pour les plus critiques d'entre eux, vérifiés automatiquement à chaque modification du code. Les principes de Security by Design ne sont pas invoqués après coup : ils se lisent dans des décisions concrètes, comme le choix de répondre 404 plutôt que 403 sur une ressource appartenant à une autre organisation, ou le rechargement de l'utilisateur à chaque requête pour qu'une désactivation prenne effet immédiatement.

Les faiblesses se situent ailleurs, et c'est le principal enseignement de cette analyse. Elles ne tiennent pas à des erreurs de programmation mais à des **dispositifs périphériques non encore mis en place** : le chiffrement du transport, qui relève du déploiement ; la limitation du nombre de tentatives, qui relève d'une fonction transverse ; la journalisation des événements de sécurité et le plan de sauvegarde, qui relèvent de l'exploitation. Une application dont le code est correct reste vulnérable si ces dispositifs manquent — et c'est précisément ce que l'exercice d'une réflexion critique permet de mettre en évidence.

Les onze actions du chapitre 12 représentent environ vingt-deux jours de travail, dont trois seulement pour traiter les deux risques les plus élevés. Ce déséquilibre est caractéristique : l'essentiel du gain de

sécurité s'obtient avec une faible part de l'effort total, à condition d'avoir correctement identifié les priorités.

Enfin, cette stratégie n'a de valeur que si elle est réévaluée. Trois événements devraient déclencher sa révision : une évolution du modèle d'authentification — notamment un passage du jeton vers un cookie, qui rendrait nécessaire la protection CSRF aujourd'hui écartée à juste titre ; l'ouverture de l'API à des clients tiers, qui imposerait une gestion de clés et de quotas ; et l'extension du périmètre à des données plus sensibles, qui justifierait alors l'authentification à deux facteurs et le chiffrement des données au repos.